



CC2032 Cloud Computing

Actividad:

Proyecto Parcial

Profesora:

Colchado Ruiz, Gerardo

Sección:

Teoría 1.01

INTEGRANTES	CÓDIGO
Cayllahua Hilario, Joel Modesto	202410731
Guerrero Gutierrez, Nayeli Belén	202410790
Maquera Quispe, Luis Fernando	202410621
Sánchez Terrazos, Leonardo Gabriel	202410273
Tamayo Hilario, Maria Karolay	202410766

Equipo 9

Descripción:

El proyecto Bus MVP representa una solución integral de gestión de transporte desarrollada bajo una arquitectura de microservicios desplegada en la nube de Amazon Web Services (AWS). Este sistema ha sido diseñado para abordar las necesidades críticas de las empresas de transporte modernas, proporcionando una plataforma escalable, resiliente y eficiente para la gestión de pasajeros, viajes, tickets e información histórica.

La solución implementa cinco microservicios especializados: gestión de pasajeros (ms-passengers), administración de viajes (ms-trips), sistema de tickets (ms-tickets), registro histórico (ms-history) y analíticas avanzadas (ms-analytics). Cada servicio opera de manera independiente utilizando tecnologías específicas como Python con FastAPI, Node.js con Express y Java con Spring Boot, lo que garantiza la flexibilidad tecnológica y la especialización por dominio de negocio.

El sistema incorpora un pipeline completo de datos que incluye procesos ETL (Extract, Transform, Load) para la ingesta de información desde múltiples fuentes hacia un data lake en Amazon S3. Los datos procesados son catalogados mediante AWS Glue y consultados a través de Amazon Athena, permitiendo análisis en tiempo real y generación de reportes ejecutivos. La arquitectura de datos soporta tres bases de datos especializadas: PostgreSQL para datos transaccionales, MySQL para información de tickets y MongoDB para almacenamiento de documentos no estructurados.

Objetivos

- Escalabilidad Horizontal: Implementación de arquitectura de microservicios que permite escalar componentes individualmente según la demanda
- Alta Disponibilidad: Configuración de load balancer con Nginx para distribución de carga y tolerancia a fallos
- Procesamiento de Datos: Pipeline automatizado de ETL para análisis de patrones de uso, rutas populares y comportamiento de clientes
- Integración Cloud-Native: Aprovechamiento completo de servicios AWS para cómputo (EC2), almacenamiento (S3) y análisis (Glue/Athena)
- Containerización: Despliegue mediante Docker y orquestación con Docker Compose para portabilidad y consistencia entre ambientes

La implementación exitosa de esta arquitectura demuestra la viabilidad de sistemas distribuidos en el sector transporte, proporcionando una base sólida para el crecimiento futuro y la incorporación de nuevas funcionalidades como pagos digitales, tracking en tiempo real y optimización de rutas mediante inteligencia artificial.

Tecnologías Utilizadas

Arquitectura y Orquestación

- Docker: Containerización de todos los servicios para garantizar portabilidad y consistencia
- Docker Compose: Orquestación multi-contenedor para gestión simplificada del stack completo
- Nginx: Load balancer y proxy reverso para alta disponibilidad y distribución de carga
- Microservicios Backend

- Python 3.9+ con FastAPI: Framework moderno para APIs REST de alto rendimiento
- Node.js con Express: Runtime JavaScript para servicios web escalables
- Java con Spring Boot: Framework empresarial para microservicios robustos

Bases de Datos

- PostgreSQL 13: Base de datos relacional para datos transaccionales críticos
- MySQL 8.0: Sistema de gestión de base de datos para información de tickets
- MongoDB 5.0: Base de datos NoSQL para almacenamiento de documentos flexibles

Servicios de Amazon Web Services (AWS)

- Amazon EC2: Instancias de cómputo elástico para hosting de la aplicación
- Amazon S3: Almacenamiento objeto escalable para el data lake
- AWS Glue: Servicio de catálogo de datos y ETL serverless
- Amazon Athena: Motor de consultas SQL para análisis de big data

Pipeline de Datos y Analytics

- Python con Boto3: SDK oficial de AWS para integración con servicios cloud
- Pandas: Biblioteca de análisis y manipulación de datos
- Flask: Framework web ligero para APIs de analytics
- JSON/Parquet: Formatos de datos para almacenamiento eficiente

Desarrollo y DevOps

- Git: Control de versiones distribuido
- GitHub: Repositorio remoto y colaboración en equipo
- YAML: Configuración declarativa para Docker Compose y OpenAPI
- Bash Scripts: Automatización de despliegue y configuración

Networking y Comunicación

- HTTP/HTTPS: Protocolo de comunicación entre servicios
- REST APIs: Arquitectura de servicios web stateless
- JSON: Formato de intercambio de datos ligero
- Docker Networks: Redes virtuales para comunicación inter-contenedor

Monitoreo y Observabilidad

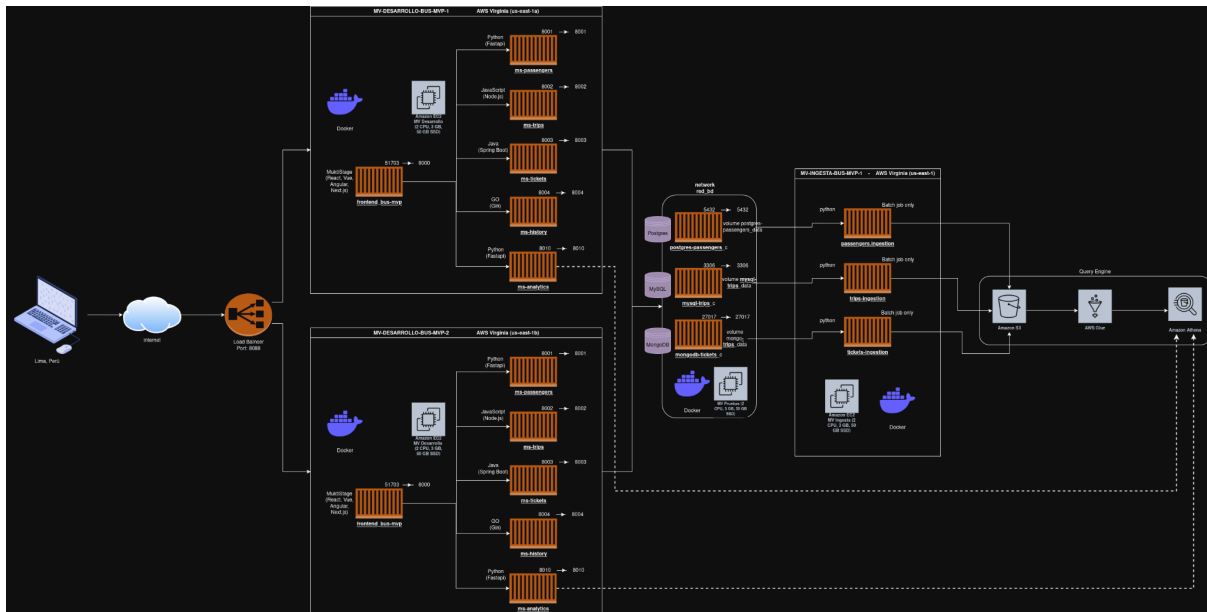
- Health Checks: Endpoints de verificación de estado de servicios
- Docker logs: Sistema de logging centralizado
- OpenAPI/Swagger: Documentación interactiva de APIs

Seguridad

- Environment Variables: Gestión segura de credenciales y configuración
- Network Isolation: Segmentación de red mediante Docker networks
- IAM Roles: Gestión de permisos granulares en AWS

Arquitectura del sistema

Diagrama de arquitectura



Link: <https://drive.google.com/file/d/1NdD2f9wcUzNeCFbv-zhyHshhIwFzBCVr/view?usp=sharing>

Arquitectura del Backend

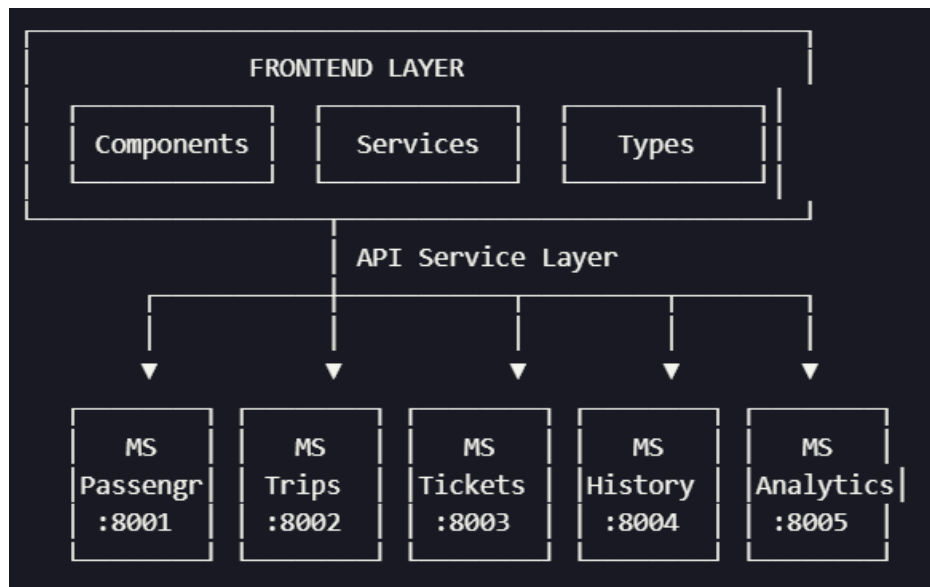
1. Estructuras y microservicios

Microservicio	Framework	Base de datos	Puerto
ms-passengers	FastAPI (Python)	PostgreSQL	8001
ms-trips	Express (Node.js)	MySQL	8002
ms-tickets	Spring Boot (Java, Maven)	MongoDB	8003
ms-history	Go (Gin)	No tiene base de datos (Consume de otros microservicios)	8004

Todos los servicios se dockerizan con su Dockerfile y se orquestan con Docker Compose con red interna, dependencias y exposición de puertos.

Arquitectura del Frontend

Portal web administrativo que integra 5 microservicios independientes, permitiendo la gestión completa del sistema de transporte de autobuses desde una interfaz unificada



1. Estructura de Archivos

```

frontend/web-portal/
├── public/           # Archivos estáticos
├── src/
│   ├── components/  # Componentes React
│   │   ├── PassengersSection.tsx
│   │   ├── TripsSection.tsx
│   │   ├── TicketsSection.tsx
│   │   ├── AnalyticsSection.tsx
│   │   ├── HistorySection.tsx
│   │   └── LoadingSpinner.tsx
│   ├── services/    # Lógica de negocio
│   │   └── api.ts    # Cliente API
│   ├── types/       # Tipos TypeScript
│   │   └── index.ts
│   ├── App.tsx      # Componente principal
│   ├── main.tsx     # Punto de entrada
│   └── index.css     # Estilos globales
├── .env             # Variables de entorno
├── package.json     # Dependencias
├── tsconfig.json    # Config TypeScript
├── vite.config.ts   # Config Vite
└── tailwind.config.js # Config Tailwind
  
```

2. Arquitectura técnica

Stack Tecnológico			
Categoría	Tecnología	Versión	Propósito
Framework	React	18.x	UI Components
Lenguaje	TypeScript	5.x	Type Safety
Build Tool	Vite	5.x	Fast Development
Routing	React Router	6.x	Navigation
Styling	Tailwind CSS	3.x	Utility-first CSS
Icons	Lucide React	-	Modern Icons
HTTP Client	Fetch API	Native	API Calls